From Trees to Tables: How Modern Databases Really Work

A Comprehensive Study of Data Models, Storage Internals, and Indexing Mechanisms

Sribatsha Dash

Research Paper

April 2026

Table of Contents

Abstract

1. Introduction

2. Hierarchical Model

2.1 Conceptual Description

2.2 Tree-Shaped Data Structure

3. Relational Model

3.1 Conceptual Description

3.2 Table-Based Data Representation

4. Data Model vs. Data Structure

5. Internal Storage in Relational Databases

5.1 Pages and Rows

5.2 Page Layout

6. Slotted Pages

6.1 Motivation

6.2 Slotted Page Structure

7. B-Tree Indexing

7.1 Role of B-Trees

7.2 B-Tree Structure

8. Querying and Access Paths

9. Concurrency and MVCC

9.1 MVCC Intuition

9.2 MVCC Timeline

10. Overwrite vs. Append-Only Models

10.1 Overwrite Model

10.2 Append-Only Ledger Model

11. Discussion

12. Conclusion

References

Abstract

This paper examines the conceptual and historical evolution of database systems from the hierarchical model to the relational model, and explains how modern relational databases physically store and access data. It clarifies a common confusion in database education: tree structures were inadequate as *logical data models* for general-purpose databases, yet tree-based structures such as B-Trees remain central as *physical indexing mechanisms*. The paper also explores slotted pages, rows, pages, indexing, concurrency through Multi-Version Concurrency Control (MVCC), and the distinction between data models and data structures. Diagrams are included to provide intuitive visualizations of the main concepts.

Keywords: database systems, hierarchical model, relational model, B-Tree, slotted pages, MVCC, data models, indexing

1. Introduction

Database systems are often taught in layers that feel disconnected: historical data models, relational theory, storage internals, indexing, and concurrency control. This can lead to the mistaken impression that if trees were considered a poor model for general-purpose data, then modern databases should not use trees at all. In reality, the conflict disappears once logical modeling is separated from physical implementation. A logical model defines how data and relationships are represented conceptually, whereas a physical storage structure determines how the database stores and retrieves data efficiently.

This paper presents a structured synthesis of the conceptual and implementation layers of modern database systems, tracing the evolution from hierarchical models to relational systems and explaining the internal mechanisms that make them efficient. Through diagrams and clear explanations, we resolve the apparent contradiction between rejecting tree-shaped schemas and embracing tree-shaped indexes.

2. Hierarchical Model

2.1 Conceptual Description

The hierarchical database model organizes data in a tree structure in which each record has exactly one parent and may have multiple children. This makes the model naturally suited for one-to-many relationships such as company-to-department-to-employee or folder-to-subfolder-to-file structures.

The strength of the hierarchical model lies in its structural clarity: traversal from a root node to descendants is efficient and conceptually simple. However, the model is rigid: a child cannot naturally belong to multiple parents, which makes many-to-many relationships difficult or impossible to represent without duplication. This leads to redundancy and update anomalies in domains where entities naturally participate in several relationships, such as students and courses or mutual friendships.

2.2 Tree-Shaped Data Structure

Figure 1 illustrates a typical hierarchical tree. Each node (except the root) has exactly one parent. In this example, there is no straightforward way for `Employee 3` to belong to both `Department A` and `Department B` without duplicating the employee node.

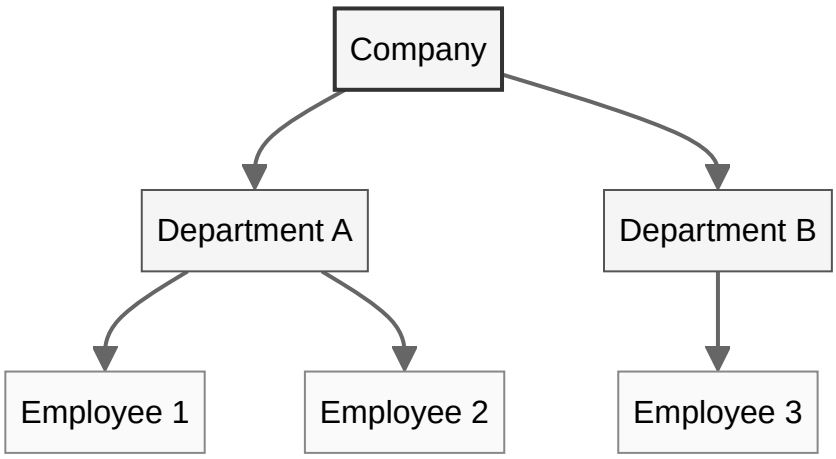

Figure 1 Hierarchical data model: a company-organized tree structure. Each employee belongs to exactly one department.

3. Relational Model

3.1 Conceptual Description

The relational model replaces structural hierarchy with tables and value-based relationships. Data is stored in relations (tables), each consisting of tuples (rows) and attributes (columns). Relationships are expressed via keys rather than via hardwired parent-child pointers.

This makes many-to-many relationships straightforward. For example, a Users table and a Courses table remain independent, while a UserCourses table records enrollment without duplicating user or course records. Queries reconstruct relationships dynamically using joins instead of following fixed tree paths.

3.2 Table-Based Data Representation

Figure 2 shows how the relational model represents entities and their relationships through independent tables connected by foreign keys.

Table 1 Users table with primary key id

id	name
1	Alice
2	Bob

Table 2 Courses table with primary key id

id	title
10	Databases
20	Networks

Table 3 UserCourses junction table implementing many-to-many mapping

user_id	course_id
1	10
1	20
2	10

Figure 2 Relational model: entities are independent tables, and relationships are defined by shared key values rather than tree position.

4. Data Model vs. Data Structure

A data model describes how data is represented conceptually: what the entities are, what attributes they have, and how they relate. A data structure describes how data is organized in memory or on disk to support efficient operations. Confusing these layers leads to misunderstandings such as "if hierarchical databases were tree-based and are now obsolete, then trees should not appear in modern databases at all."

The hierarchical model uses a tree as the *model of reality*, forcing all relationships into a parent-child shape. B-Trees, by contrast, are internal data structures used to accelerate search and lookup for arbitrary logical models. They do not impose any tree-like constraints on the user-visible schema.

Table 4 Comparison of data models and data structures

Aspect	Data Model (Logical)	Data Structure (Physical)
Purpose	Represent real-world entities and relations	Store and access data efficiently
Examples	Hierarchical, relational, graph	Array, hash table, B-Tree, slotted page
User-facing?	Yes	No (usually hidden inside the DBMS)

5. Internal Storage in Relational Databases

5.1 Pages and Rows

Internally, relational databases store table data in fixed-size pages (or blocks), not as a single contiguous array of rows. Each page is a unit of I/O, read and written from disk as a whole, and contains multiple rows plus metadata. As a table grows, it spans many pages, and the DBMS manages how rows are placed across these pages.

Rather than thinking of a table as one big array, it is more accurate to imagine a collection of pages, each using a layout that supports variable-length rows, updates, deletes, and inserts without expensive rearrangement.

5.2 Page Layout

Figure 3 shows the high-level layout of a database page. The page header contains metadata, followed by a slot directory that maps slot indices to row offsets.

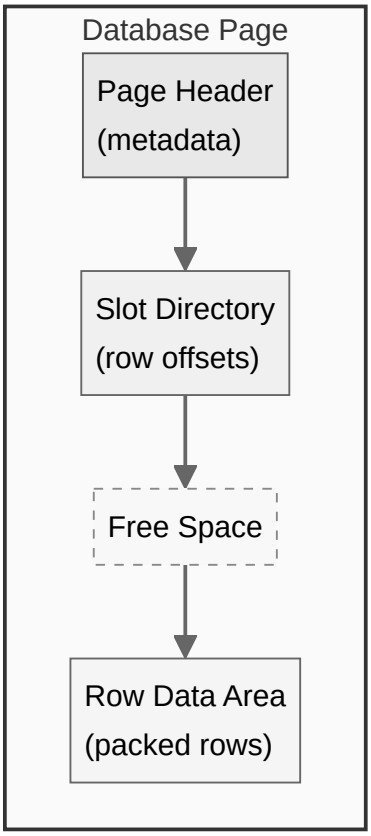

Figure 3 High-level page layout: header, slot directory, free space, and row data area.

6. Slotted Pages

6.1 Motivation

If a page stored rows in a simple array, deleting or updating rows could require shifting many entries, which would be expensive. Variable-length rows would also make offsets unstable, and any external references based on position would break easily. Slotted pages solve this by introducing a level of indirection.

6.2 Slotted Page Structure

Figure 4 illustrates the slotted page architecture. Slots are small fixed-size entries that map slot indices to row offsets. Rows themselves are stored at the "end" of the page and can move around

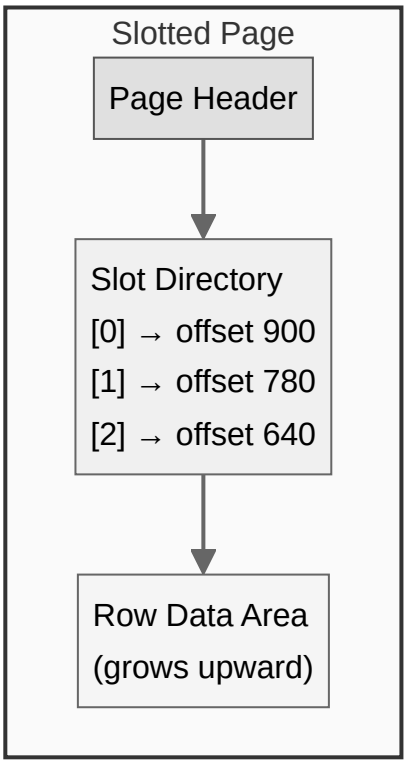

Figure 4 Slotted page structure: slot directory maps logical row IDs to physical offsets. Rows can be relocated without changing their logical identifiers.

7. B-Tree Indexing

7.1 Role of B-Trees

B-Trees (and their variants like B+ Trees) are multi-way search trees optimized for disk-based storage. They allow the DBMS to find rows matching a key (such as a primary key) with a small number of page reads. Importantly, B-Trees are used as *index structures*, not as the primary logical representation of entities and relationships.

A B-Tree node holds many keys and child pointers, which keeps the tree shallow even for large datasets. Searching involves descending from the root to a leaf, at each level choosing the child whose key range contains the target value.

7.2 B-Tree Structure

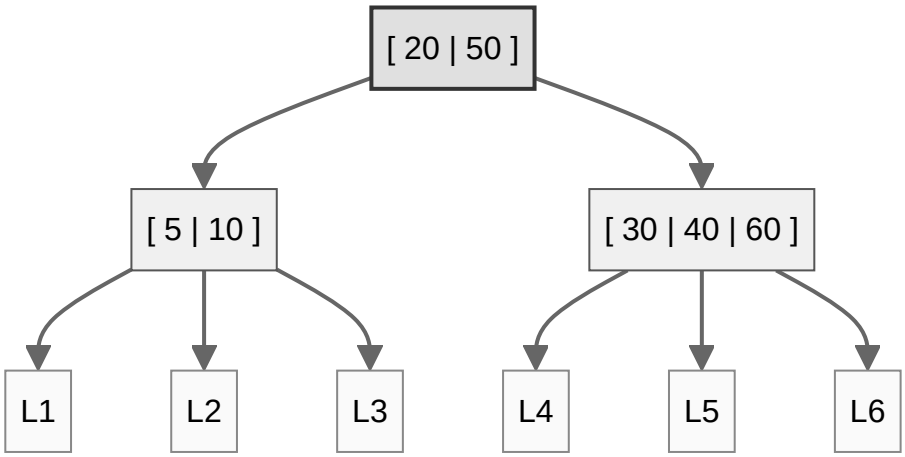

Figure 5 Conceptual B-Tree index: internal nodes store key separators; leaf nodes (L1-L6) store pointers to actual row data.

8. Querying and Access Paths

When a query references a predicate on an indexed column, the DBMS may use the corresponding B-Tree to locate relevant key values, then follow the associated row identifiers to fetch pages and interpret row data. Without an index, the system must fall back to scanning pages sequentially. The logical operation (such as joining Users and Orders on a key) remains the same; what changes is the physical strategy used to find the needed rows.

The presence of indexes does not turn the database into a "tree database." The relational model remains intact at the logical level, while tree structures operate internally to optimize access paths for particular predicates.

9. Concurrency and MVCC

9.1 MVCC Intuition

Multi-Version Concurrency Control (MVCC) allows readers to see a consistent snapshot of the database without being blocked by concurrent writers. Instead of overwriting a row in place, a write operation creates a new version of the row while older versions remain visible to transactions that started earlier. This separates *visibility* from *physical storage* and reduces read-write conflicts.

Crucially, MVCC does more than show old versions. When a transaction that read an older version attempts to update a row, the DBMS must check whether the data it read is still current. If another transaction has already committed a newer version, the late transaction may be aborted and forced to retry based on the latest committed state.

9.2 MVCC Timeline

Figure 6 illustrates the MVCC protocol in action. Transaction T1 reads a balance of 1000 and computes a new value of 700. Meanwhile, transaction T2 reads the same balance, writes a new value of 1500, and commits. When T1 attempts to commit, the DBMS detects that the current committed value (1500) differs from what T1 originally read (1000), and aborts T1.

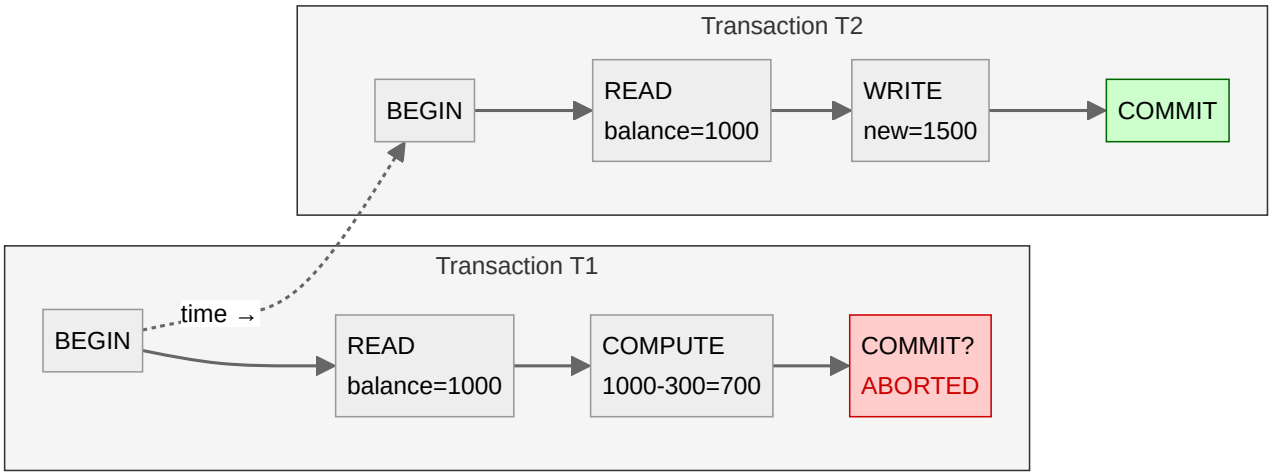

Figure 6 MVCC timeline: T1 is aborted at commit time because its read snapshot (1000) is stale relative to T2's committed write (1500).

10. Overwrite vs. Append-Only Models

10.1 Overwrite Model

In a traditional overwrite model, an account balance might be stored as a single value that is updated in place. Concurrent updates then require careful concurrency control, such as MVCC or locking, to avoid lost updates or inconsistent results.

10.2 Append-Only Ledger Model

An alternative is to record each change as an independent event (for example, +500 deposit, -300 withdrawal) in an append-only ledger, and derive the current balance by summing events. This avoids direct overwrites of the same value, but practical systems still maintain a materialized balance for quick reads and rely on atomic updates or transactions when applying new events.

Table 5 Append-only transaction ledger

tx_id	account	amount
1	A	+1000
2	A	+500
3	A	-300

Table 6 Derived balance table (1000 + 500 - 300 = 1200)

account	balance
A	1200

Figure 7 Ledger model: current state is derived from a history of append-only events.

Even in this model, the update of the balance table must be coordinated with the insertion of new transactions using transactional mechanisms to avoid race conditions, especially under high concurrency.

11. Discussion

The historical shift from hierarchical to relational databases can be misunderstood if the distinction between data model and data structure is not made explicit. Hierarchical databases failed as a *general-purpose model* because real-world relationships frequently require many-to-many connections and flexible querying beyond what a rigid tree can represent. Relational databases solved this by separating entities, attributes, and relationships into tables with key-based connections.

At the same time, relational systems embrace tree-based structures at the implementation level. Slotted pages organize row storage within pages, and B-Tree indexes provide fast search over

keys. MVCC and transaction logic allow safe concurrent updates over both overwrite-style and append-only representations. The diagrams in this paper make it clear that there is no contradiction between rejecting a tree-shaped *schema* and embracing tree-shaped *indexes*.

12. Conclusion

Database systems evolved from rigid structural models toward flexible logical representations while simultaneously developing powerful internal storage and indexing mechanisms. The hierarchical model demonstrated the usefulness of tree-shaped organization for some domains, but its limitations with complex relationships made it unsuitable as a universal model. The relational model overcame this by modeling entities and relationships via tables and keys, while leaving physical storage and access paths to specialized data structures.

Modern databases therefore use trees and related structures extensively at the physical level (for example, B-Trees and slotted pages) while keeping the logical model relational. MVCC and ledger-style patterns further extend this design to handle concurrency and correctness. Viewing these layers together, with the aid of diagrams, provides a coherent picture of how historical data models connect to real database internals.

References

[1] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

[3] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.

[4] Graefe, G. (2011). Modern B-Tree techniques. *Foundations and Trends in Databases*, 3(4), 203-402.

[5] Lehman, P. L., & Yao, S. B. (1981). Efficient locking for concurrent operations on B-Trees. *ACM Transactions on Database Systems*, 6(4), 650-670.

[6] Bernstein, P. A., & Goodman, N. (1983). Multiversion concurrency control: Theory and algorithms. *ACM Transactions on Database Systems*, 8(4), 465-483.